

Bsides Vancouver 2026

Web App Security 101 Workshop

Class:
Special Topics

Workshop Number:
AS-SPC-1337_26

Document Version:
1.00

Special Requirements:
Registered account at
picoctf.org



Welcome to the Workshop!

In this interactive workshop, we'll be learning some fundamental skills used in web app security via task-based exercises

The workshop will be divided into the following sections:



Workshop Topics and Schedule

1:00 – 1:30	Pt 1 Viewing Page Source
1:30 – 2:00	Pt 2 Client-Side Secrets
2:00 – 2:30	Pt 3 Robots.txt
2:30 – 3:00	Pt 4 Cookie Security
3:00 – 3:20	Break Time
3:20 – 3:50	Pt 5 Header Injection
3:50 – 4:20	Pt 6 SQL Injection Intro
4:20 – 4:50	Pt 7 Capstone Challenges
4:50 – 5:00	Wrap-Up

The Master Workshop Document

[https://github.com/theshyhat/DC604
/blob/main/web101/notes.md](https://github.com/theshyhat/DC604/blob/main/web101/notes.md)

The document with all the links we'll be using today can be found at the link above

PicoCTF / CyLab Platform



CyLab

SECURITY ACADEMY

To get hands-on with the concepts we're learning today, we'll be solving challenges from PicoCTF, which has recently rebranded to CyLab Security Academy

PicoCTF / CyLab Platform



CyLab
SECURITY ACADEMY

If you have an existing account at picoCTF, you can use the same username / password to access the CyLab website

PicoCTF / CyLab Platform



CyLab

SECURITY ACADEMY

If you don't have an account, you'll need to make one. The signup link can be found in the Master Workshop document under the Pre-Start Links section

Firefox Web Browser



The learn some of the concepts we'll be learning today, we should use the Firefox web browser, because it has built-in tools that aren't available to other popular browsers, like Chrome

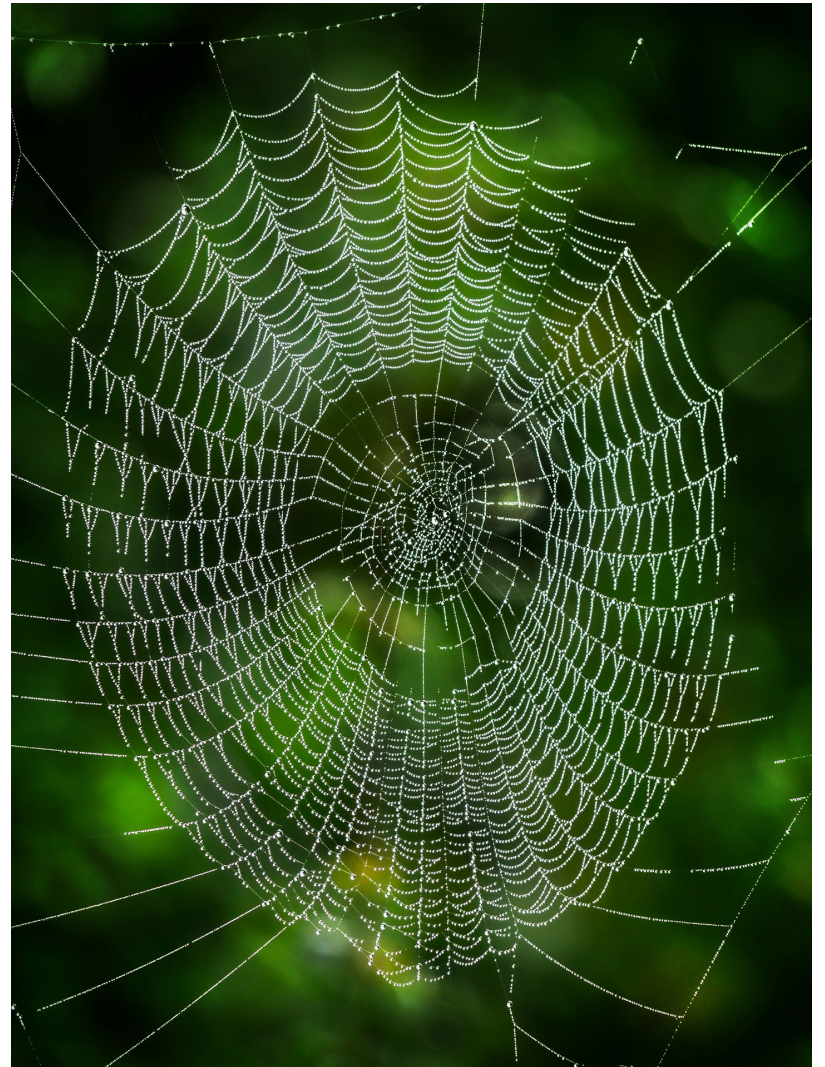
Firefox Web Browser



If you don't have Firefox installed, you can find the download link on the Master Workshop document, under the Pre-Start Links section at the top

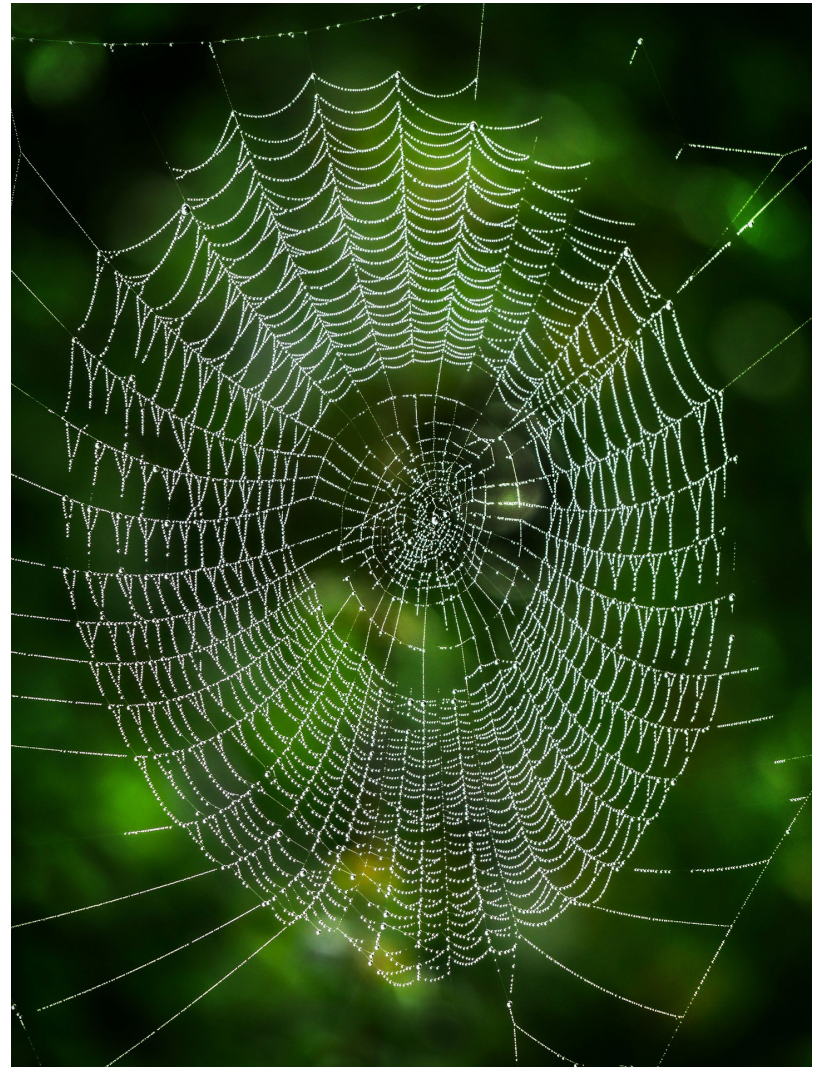
What is Web App Hacking?

Web app hacking is the intentional abuse of systems and applications accessed via the internet by a user, which benefits that user in some way.



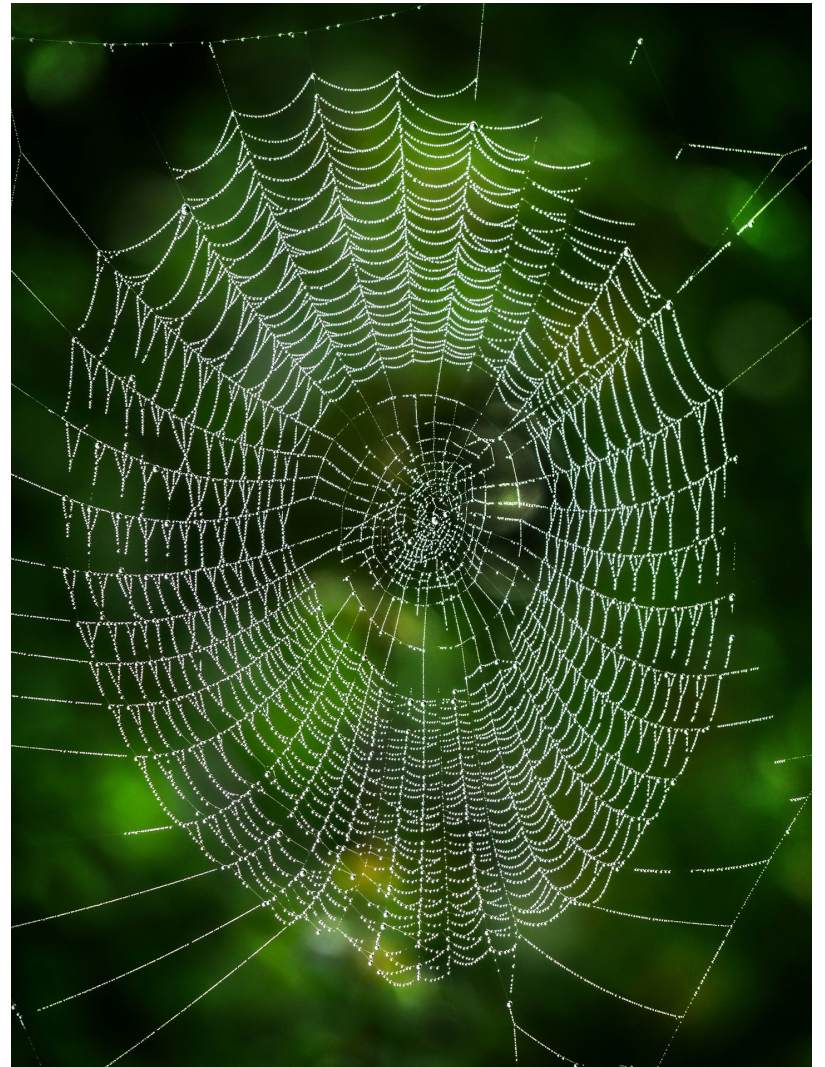
What is Web App Hacking?

This benefit can take the form of access to unauthorized data or systems, the deletion of data, denial of website services, or any number of other benefits.



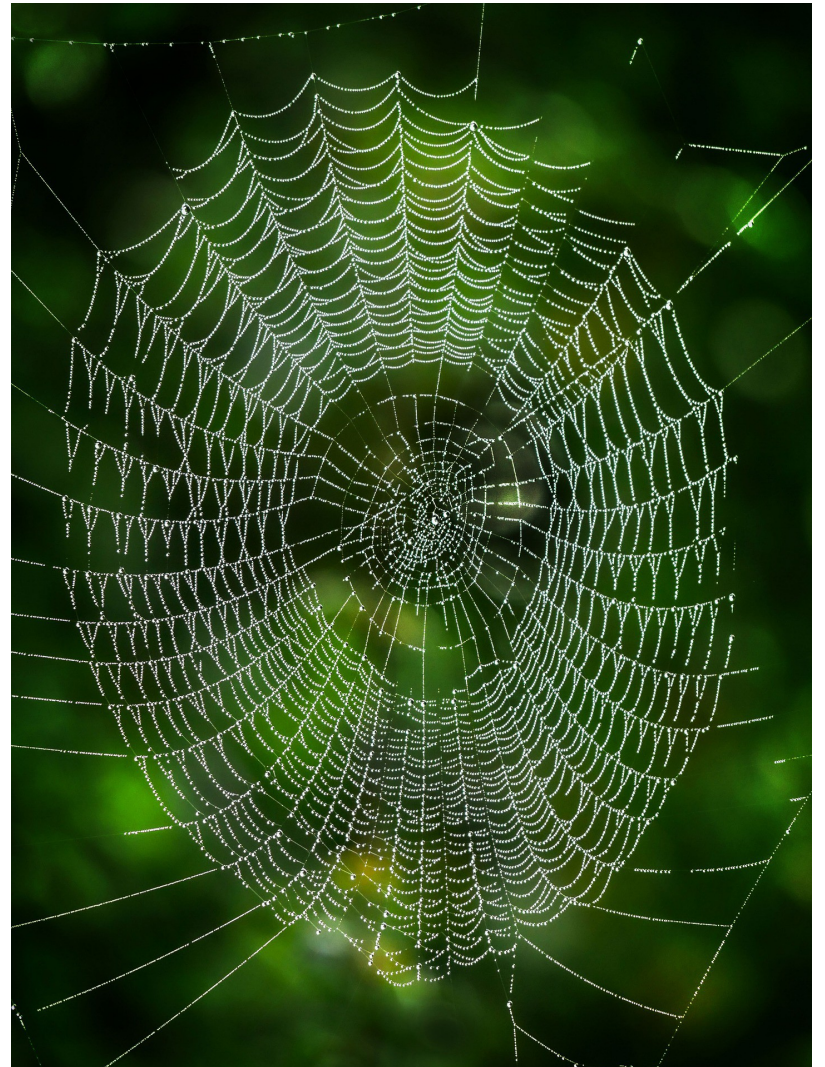
What is Web App Hacking?

In order to learn web app hacking, we must learn about the internet, how websites work, and a good amount about the technologies and software found in that space.



What is Web App Hacking?

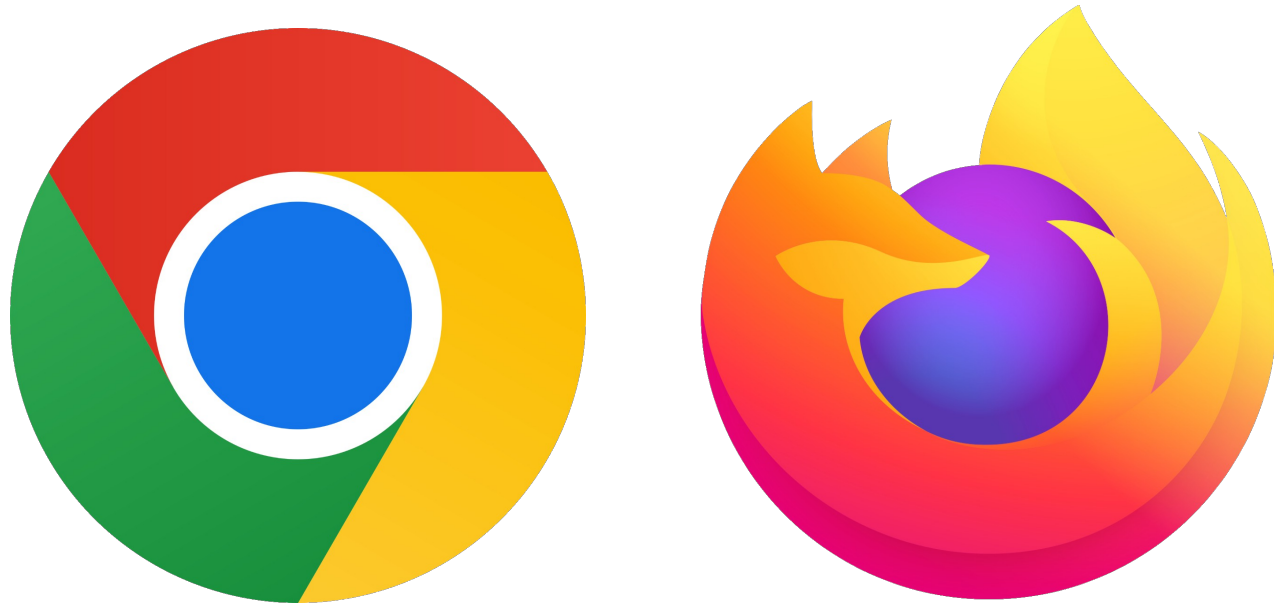
Most students know at least the basics of using a web browser and how to use websites, so we will build on those skills to establish a collection of techniques to explore and hack web applications found on the internet.



Part 1 – Viewing Page Source

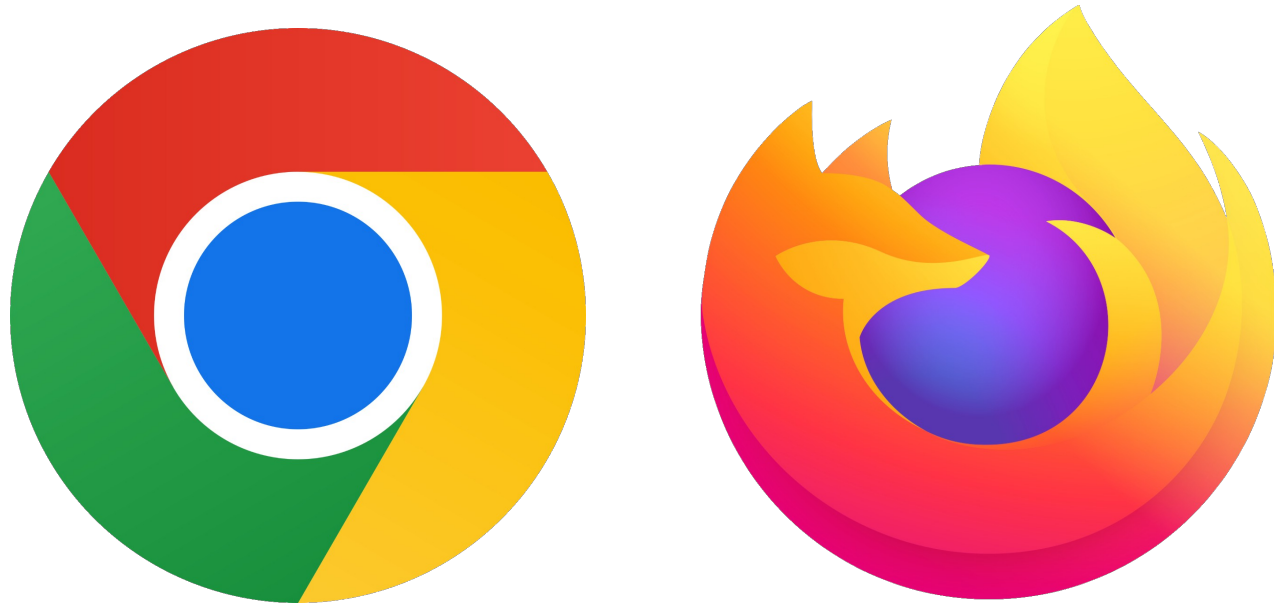
```
1 <!-- This Source Code Form is subject to the terms of the Mozilla Public
2    - License, v. 2.0. If a copy of the MPL was not distributed with this file,
3    - You can obtain one at http://mozilla.org/MPL/2.0/. -->
4
5 <!DOCTYPE html>
6 <html>
7   <head>
8     <meta charset="utf-8" />
9     <meta
10       http-equiv="Content-Security-Policy"
11       content="default-src 'none'; object-src 'none'; script-src resource: chrome;;
12       blob: chrome;; style-src 'unsafe-inline';"
13     />
14     <meta name="color-scheme" content="light dark" />
15     <title data-l10n-id="newtab-page-title"></title>
16     <link
```

What is a Web Browser?



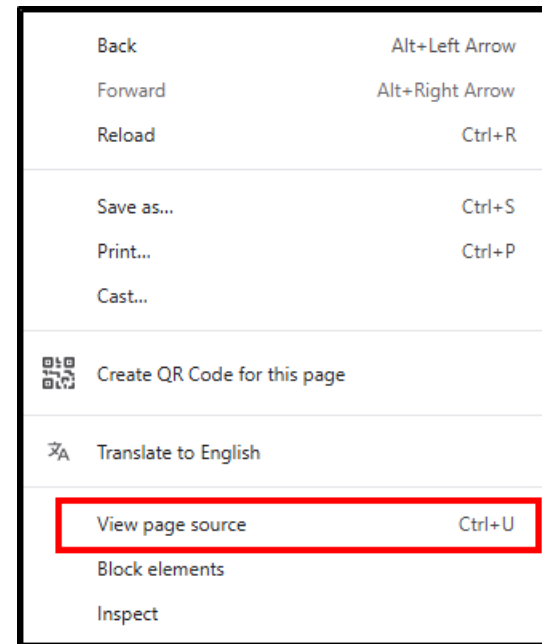
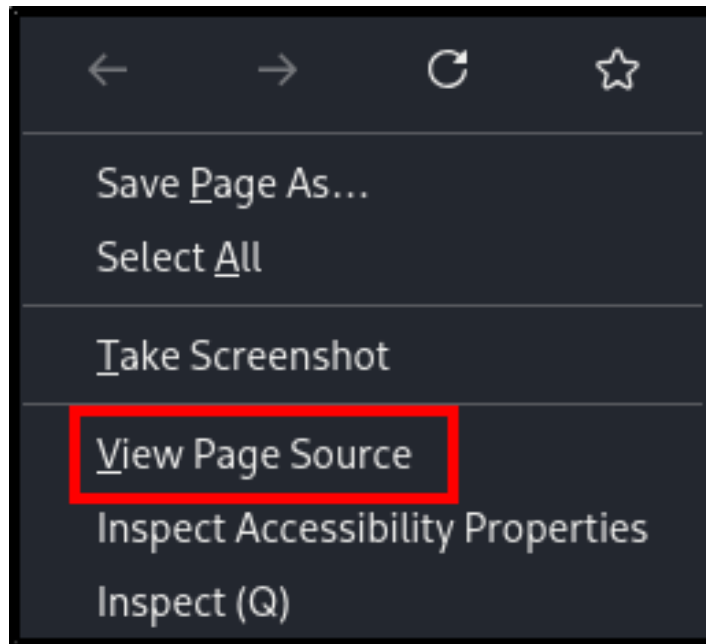
A web browser is a program which receives web page code and displays it to the user

What is a Web Browser?



Most web browsers allow the user to look at the source code of any particular webpage

How Do We Look at Page Source?



Accessing the source code for a webpage in browser usually involves right-clicking and selecting "View Page Source"

What Can We Find in Page Source?

```
<link
  rel="icon"
  type="image/png"
  href="chrome://branding/content/icon32.png"
/>
<link rel="localization" href="branding/brand.ftl" />
<link rel="localization" href="toolkit/branding/brandings.ftl" />
<link rel="localization" href="browser/newtab/newtab.ftl" />
<link
  rel="stylesheet"
  href="chrome://global/skin/design-system/tokens-brand.css"
/>
```

In a webpage's source code, we can see all of the other files that the webpage uses (its assets)

What Can We Find in Page Source?

You can find the password for the next level on this page.

```
<!--The password for natas1 is d0u5t1p4q7c271d4p4d9h0w17h49t1k1w1q -->  
</div>
```

Another interesting thing we can see in web page source are comments left by the developers, which can sometimes contain sensitive info

What are CTFs?

This workshop incorporates CTF games into the topics for a more interactive experience, but what are CTFs?



What are CTFs?

Cybersecurity Capture The Flag (CTF) games are training exercises where the goal of the exercise is to “capture the flag” through use of cybersecurity skills.



What are CTFs?

In this context, “capture” means to gain access to a file or other resource, and “flag” refers to a secret phrase or password.



CyLab Security Academy



The CTF game we will be playing to learn basic web app hacking skills is called Cylab. It's a well-known education platform with challenges across many different cybersecurity disciplines.

First Challenges: Inspect HTML, Unminify, and Includes

In the master workshop document under Topic 1, try to solve the three challenges, **Inspect HTML**, **Unminify**, and **Includes**

Don't worry if you can't finish them all. We'll walk through the challenges afterwards

Part 2 – Client-Side Secrets



What Are Client-Side Secrets?



Client-Side Secrets are sensitive information, (in the form of comments, code, or assets) exposed to the client-side of the web application

What Are Client-Side Secrets?



Client-side, in this case refers to anything that can be accessed by the web browser

Examples of Client-Side Secrets

```
</style>  
<!-- Email: guest@picoctf.org Password: guest -->  
</head>  
<body>
```

One example of a client-side secret could be a set of credentials exposed in a web page comment

Examples of Client-Side Secrets

```
function checkPassword(username, password)
{
  if( username === 'admin' && password === 'strongPassword998765' )
  {
    return true;
  }
  else
  {
    return false;
  }
}
```

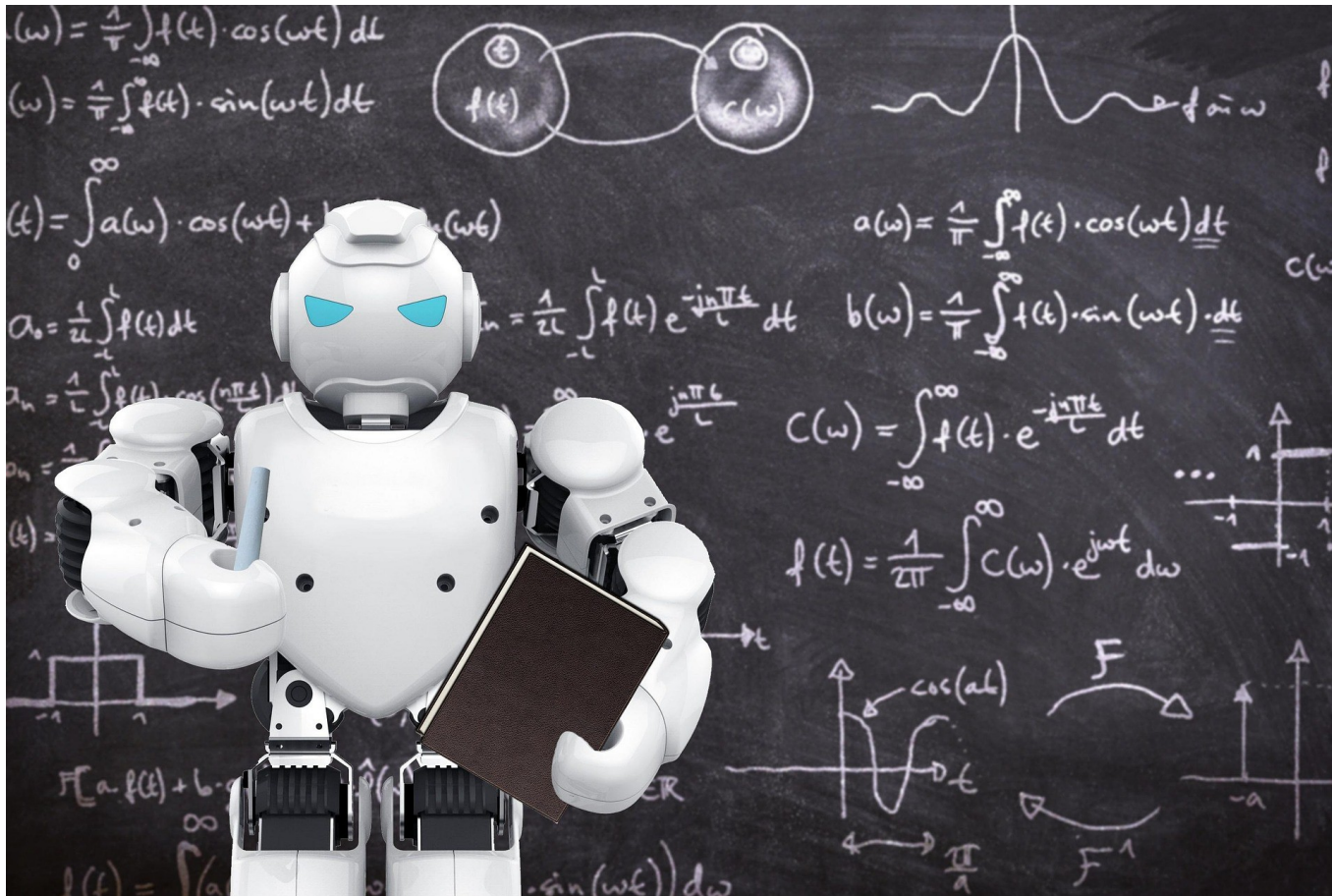
Another client-side secret could be a security / login function exposed in a client-side asset

Challenges: Don't Use Client Side and Local Authority

The next challenges we'll cover this session is called **Don't Use Client Side**, and **Local Authority** which can be accessed from the master document in the Part 2 section

Hint: Check out all of the links in the web page source and look for code that handles login, passwords, or other types of security

Part 3 – Robots.txt and Data Obfuscation



Common Data Obfuscation Techniques

980 874 2903 0 9874 7 288 841 0 28 2749 83658 B9 20 89B 650 21 83BN 9886 X 686
928098 7408 80898 3283 0023 897 833 9283 492N 8 2398 49384 29824 6525719
889 834 4M 982 00438 47838 4 K92.83 9281 0656 5678 857 2116 065 676 0365372
83563672 86542 2837 29987429 284 038 7M 65836 265876 837 0M 827 468872
46M 63882 864823 8438612 065623 7628 8 0640 P028738 6433 834768 8284
47M 935 083748 872466 658767 0 878738 38 07648 2734621 00823 64 86404787
09 884 8874 788 7410 9284 780 868 8064 837 048 6740 28370 528 0708 0684637
2418874 092383 66437 6264 837 048 827428 7488 846887 8687321 0388 468786
506527380 846738 2876847 864 287364 88746 2888 091082 76473 07048386
980 874 2903 0 9874 7 288 841 0 28 2749 83658 B9 20 89B 650 21 83BN 9886 X 686
928098 7408 80898 3283 0023 897 833 9283 492N 8 2398 49384 29824 6525719
889 834 4M 982 00438 47838 4 K92.83 9281 0656 5678 857 2116 065 676 0365372
83563672 86542 2837 29987429 284 038 7M 65836 265876 837 0M 827 468872
46M 63882 864823 8438612 065623 7628 8 0640 P028738 6433 834768 8284

In CTF exercises, you will often encounter encoded / encrypted information which will need to be decoded / decrypted to understand

Common Data Obfuscation Techniques



980 P74 290B 0 9874 7 288 N41 0 28 2 749 8 3658 B9 20 89B 6 60 2 1 8 3BN 988 6 X 686
928 998 7408 8 88 98 3 283 002B 897 8 33 9 283 492N 8 2398 4 9384 2 9834 6 5 2 5 7 1 9
889 8 34 4 4 982 0 0 4 38 4 7 3 8 4 K92 8 3 9 28 1 0 656 5 6 7 8 5 7 2 1 6 0 6 5 6 7 6 0 3 6 5 3 7 2
8 3 6 6 3 6 7 2 8 6 5 4 2 2 8 3 7 2 9 9 8 7 4 2 9 2 8 4 0 3 8 7 4 6 5 8 3 6 2 6 5 8 7 6 8 3 7 0 1 8 2 7 4 6 8 8 7 4
4 6 4 6 3 8 8 2 8 6 4 8 2 3 8 4 3 8 6 1 2 0 6 5 6 2 3 7 6 2 8 8 7 6 4 0 P 0 2 8 7 3 4 8 6 4 3 7 8 3 4 7 6 8 8 2 8 4
4 7 4 9 3 5 0 8 3 7 4 8 3 7 2 4 6 6 6 5 6 7 6 7 6 8 7 8 7 3 8 3 8 2 7 6 4 8 2 7 3 4 6 2 1 0 0 8 2 3 6 4 8 6 4 0 4 3 7
0 9 8 8 4 8 8 7 4 7 8 8 7 4 1 0 9 2 8 4 7 0 8 6 8 8 0 6 4 8 8 7 6 4 8 6 7 4 0 2 8 3 7 6 5 2 8 7 1 0 8 0 6 4 6 3 7
2 4 1 3 8 7 4 0 2 3 8 6 6 6 4 3 7 6 2 6 4 8 3 7 0 1 8 8 2 1 4 2 8 7 4 8 8 8 4 6 8 8 7 6 8 3 7 3 2 1 0 1 8 8 4 6 8 7 8 6
5 4 6 5 2 7 3 8 0 3 4 6 7 3 8 2 8 7 6 7 4 7 3 6 4 2 8 7 3 6 4 8 7 4 6 2 8 8 8 0 9 1 0 8 2 7 6 4 7 3 0 7 0 4 8 3 6
980 P74 290B 0 9874 7 288 N41 0 28 2 749 8 3658 B9 20 89B 6 60 2 1 8 3BN 988 6 X 686
928 998 7408 8 88 98 3 283 002B 897 8 33 9 283 492N 8 2398 4 9384 2 9834 6 5 2 5 7 1 9
889 8 34 4 4 982 0 0 4 38 4 7 3 8 4 K92 8 3 9 28 1 0 656 5 6 7 8 5 7 2 1 6 0 6 5 6 7 6 0 3 6 5 3 7 2
8 3 6 6 3 6 7 2 8 6 5 4 2 2 8 3 7 2 9 9 8 7 4 2 9 2 8 4 0 3 8 7 4 6 5 8 3 6 2 6 5 8 7 6 8 3 7 0 1 8 2 7 4 6 8 8 7 4
4 6 4 6 3 8 8 2 8 6 4 8 2 3 8 4 3 8 6 1 2 0 6 5 6 2 3 7 6 2 8 8 7 6 4 0 P 0 2 8 7 3 4 8 6 4 3 7 8 3 4 7 6 8 8 2 8 4

The two most-commonly encountered obfuscation techniques are **Base64** encoding and **ROT13** encryption

Base64 Encoding

hello → aGVsbG8=

hacking is fun → aGFja2luZyBpcyBmdW4h

1234567980 → MTIzNDU2Nzg5MA==

Base64 is an encoding system which transforms data into an alphanumeric string that may include a forward slash or a plus symbol

Base64 Encoding

hello → aGVsbG8=

hacking is fun → aGFja2luZyBpcyBmdW4h

1234567980 → MTIzNDU2Nzg5MA==

Another obvious sign that some data is base64 encoded is the equal signs at the end of the strings

ROT13 Encryption

A B C D E F G H I J K L M

| | | | | | | | | | | |

N O P Q R S T U V W X Y X

The other commonly-seen obfuscation method in CTFs is ROT13, which is an application of a classical cryptographic cipher

ROT13 Encryption

A B C D E F G H I J K L M

| | | | | | | | | | | |

N O P Q R S T U V W X Y X

ROT13 can be thought of to mean “rotate 13 letters”, since the method of encrypting or decrypting strings using ROT13 is--

ROT13 Encryption

hello ↔ uryyb

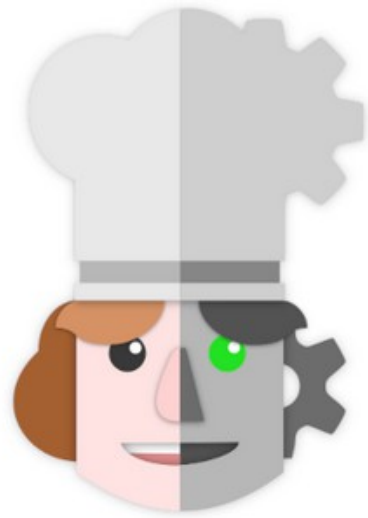
To rotate each letter in the message forward 13 characters to form the secret (ciphertext)

ROT13 Encryption

hello ↔ uryyb

To transform the secret back into its original form (plaintext), we rotate the letters in the message by 13 letters again

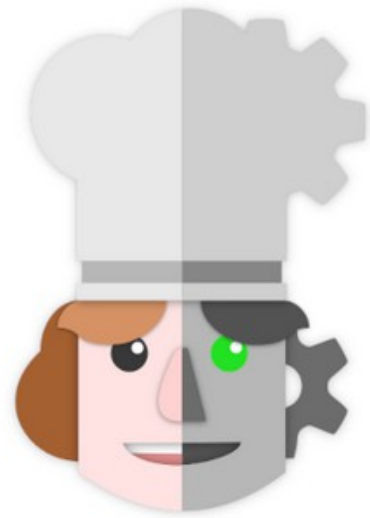
Web Tools For Deciphering



CyberChef

There are two links for deciphering strings from Base64 and ROT13 in the master document in the Part 3 section, using the CyberChef tool

Web Tools For Deciphering

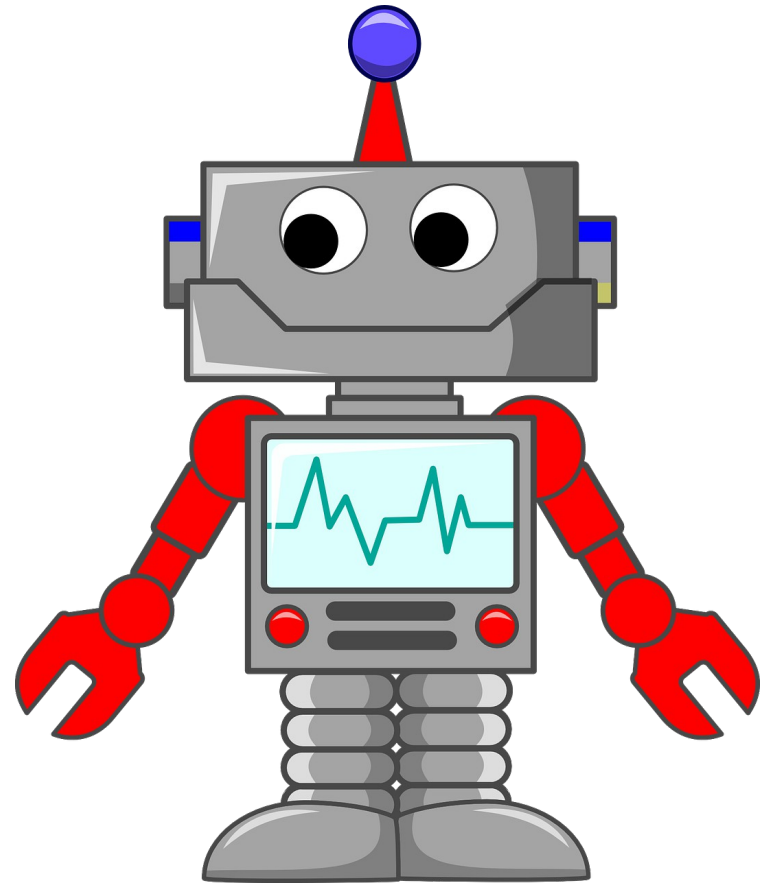


CyberChef

Let's try deciphering a few strings for practice

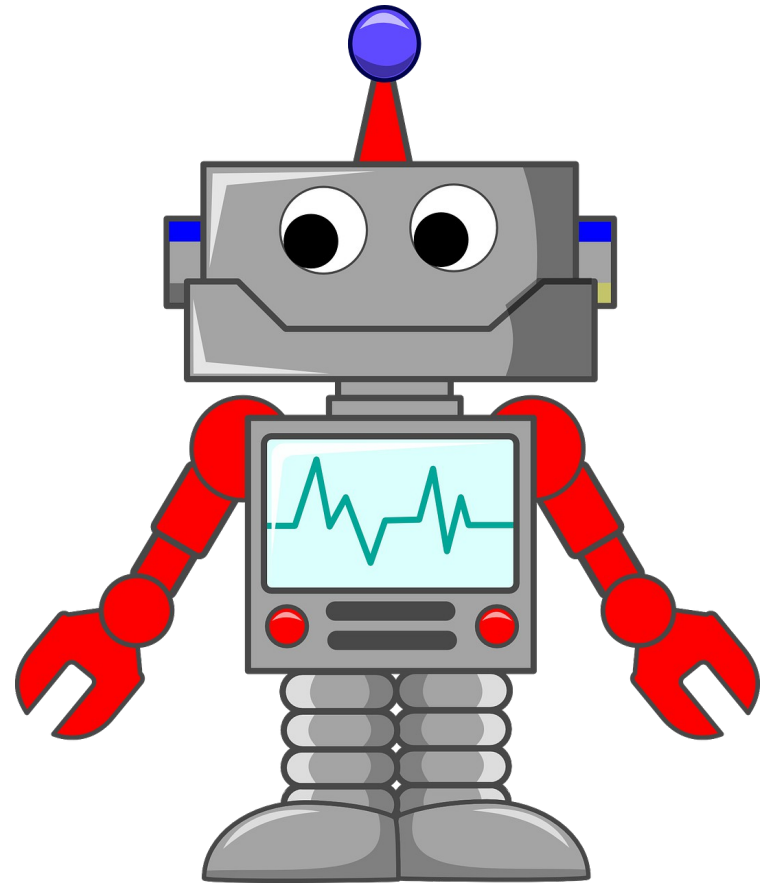
What is the Robots.txt File?

Search engines (such as Google, Yahoo, DuckDuckGo, etc) use programs called robots to visit websites and map out their webpages. However, this may cause sensitive areas of websites to appear in search results.



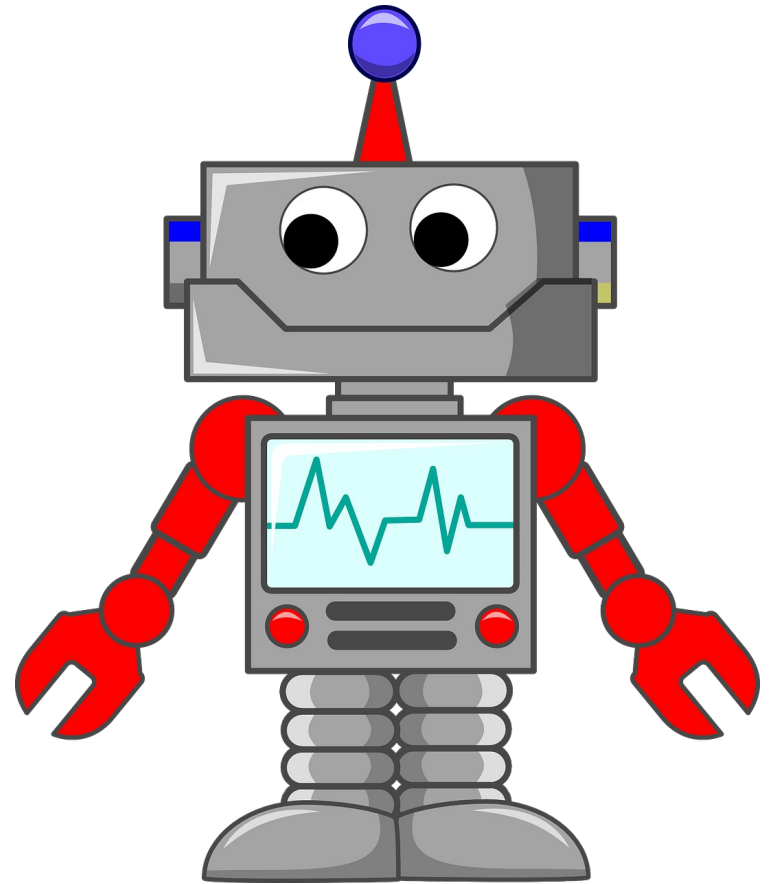
What is the Robots.txt File?

In order to prevent this, website administrators can add a file called **robots.txt** to their website, which specifies which directories and / or pages of the website are off-limits to search engine robot programs.



What is the Robots.txt File?

Unfortunately, if malicious users know how to find the **robots.txt** file, the contents of the file could potentially lead them to sensitive areas of the website.



Challenges: Where Are The Robots? and Roboto Sans

The next challenges we'll cover this session is called **Where Are the Robots?**, and **Roboto Sans**, which can be accessed from the master document in the Part 3 section

Hint: Remember to keep a lookout for obfuscated data, like ROT13 and Base64-encoded strings

Part 4: Cookie Security



What Are Cookies?

Cookies are used to retain user settings or establish / maintain a user session on a website.

They're a very common feature of web applications



What Are Cookies?

For example, a website has a button on its user preferences page which sets the webpage color scheme, light mode or dark mode.



What Are Cookies?

Once the theme is selected, the web server will send a Cookie to the web browser to be used anytime the website is visited, changing the webpage's color theme to whatever is specified in the Cookie.



What Are Cookies?

Similarly, when a user successfully logs into a website, the web server will send the web browser a Cookie that identifies which user session is being used, and the browser will use that Cookie each time that website is accessed.

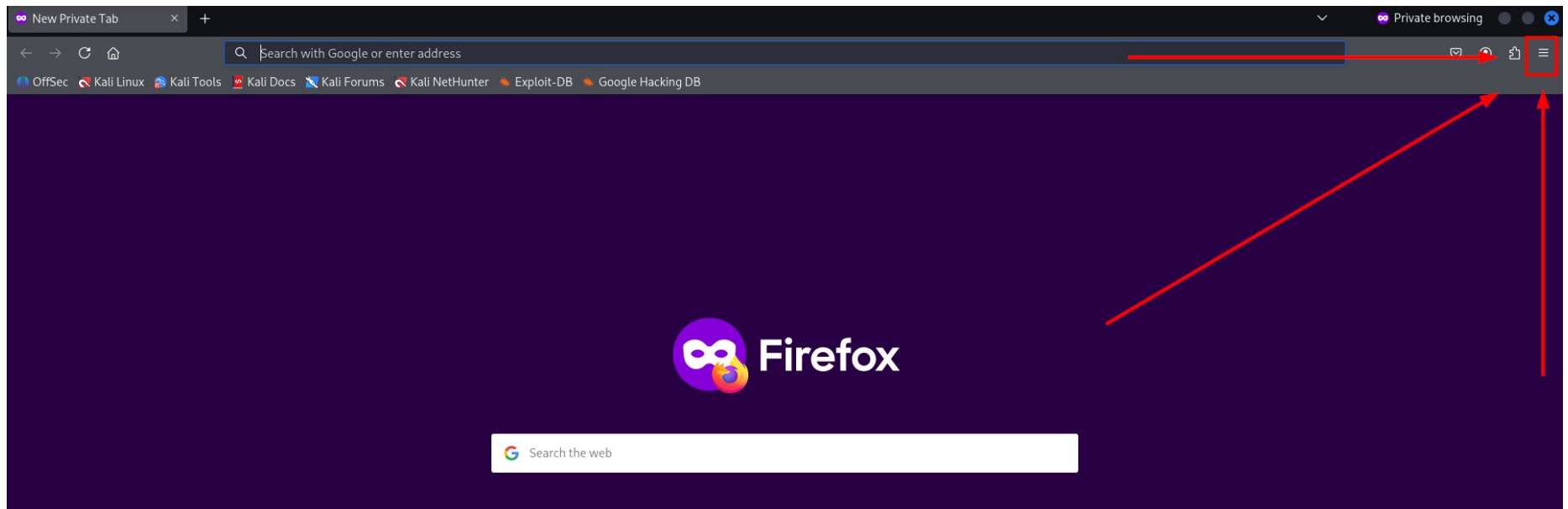


What Are Cookies?

Any Cookie that is used for user sessions has the potential for security abuse, so it is important that the Cookie values created for user sessions are not predictable at all.



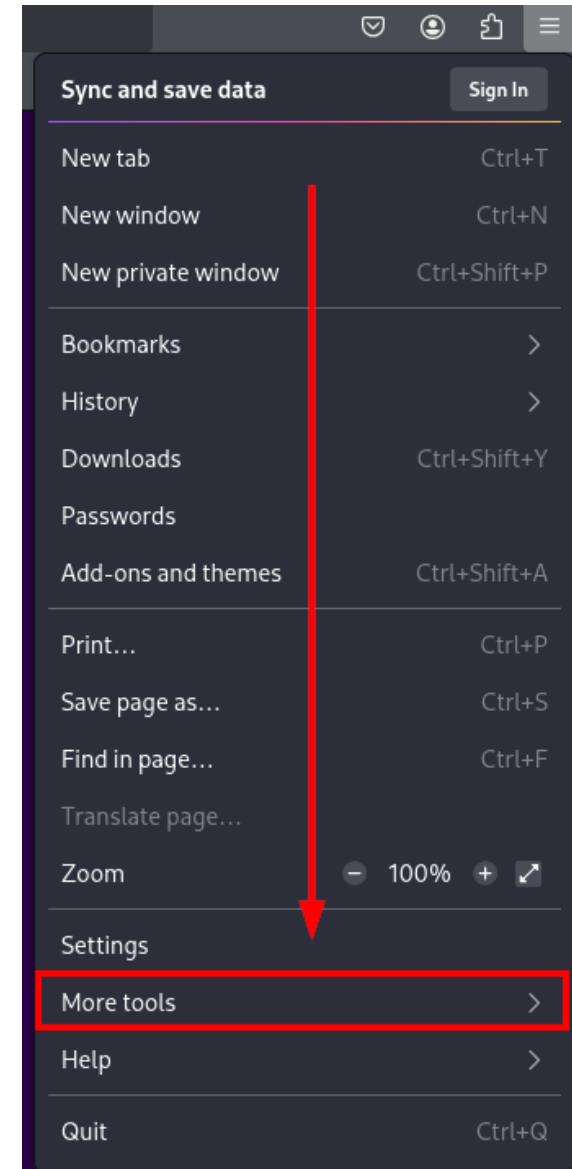
How Do We Access Cookies?



Cookies can be accessed in the Firefox web browser by clicking on the "hamburger menu" at the top-right of the browser window

How Do We Access Cookies?

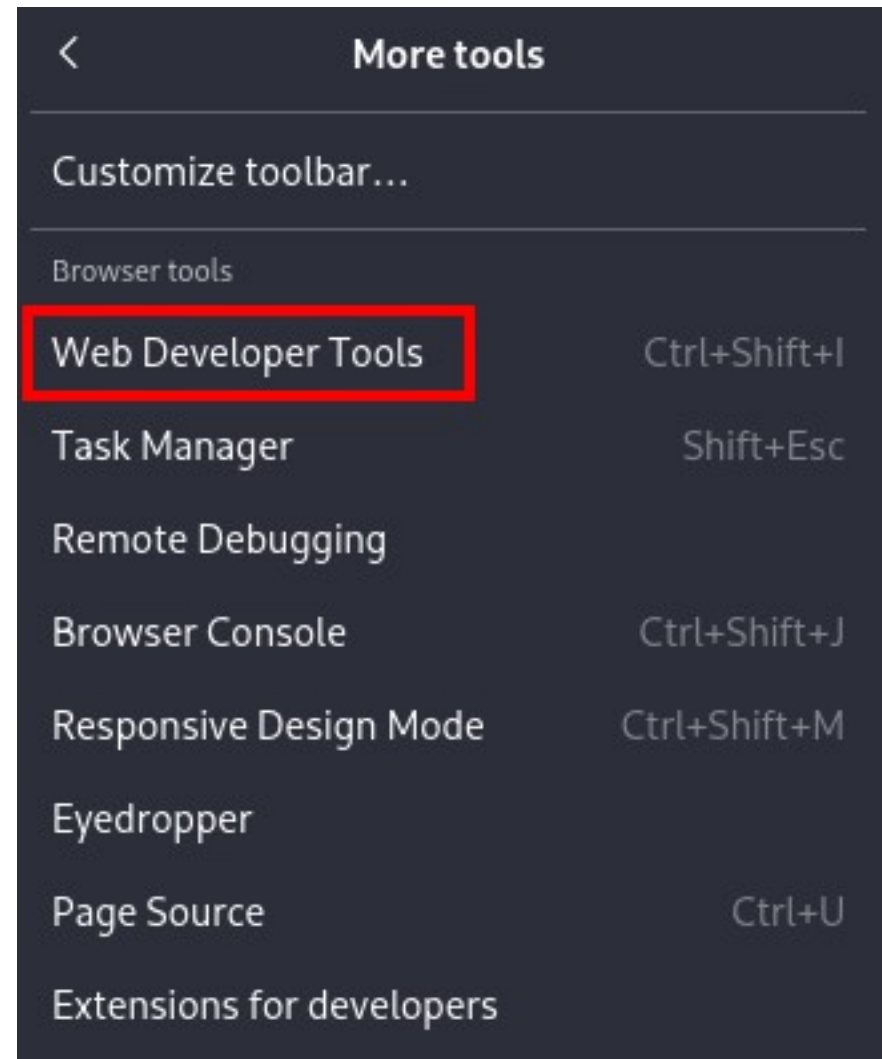
Then click on More Tools



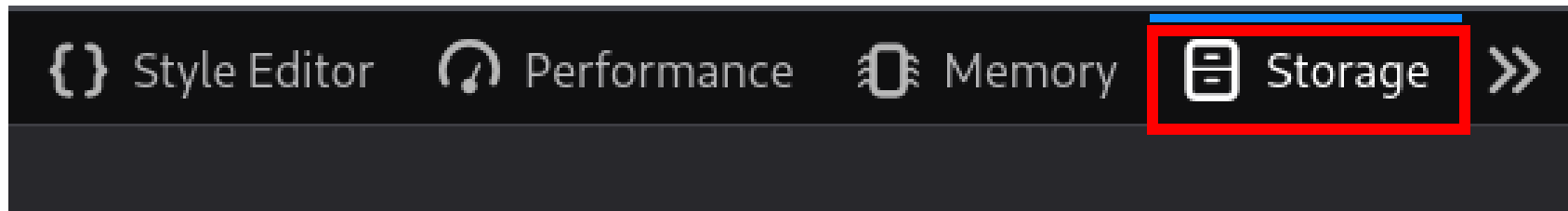
How Do We Access Cookies?

Then click on
Web Developer Tools

If you want to use
keyboard shortcuts:
F12 or Ctrl+Shift+I

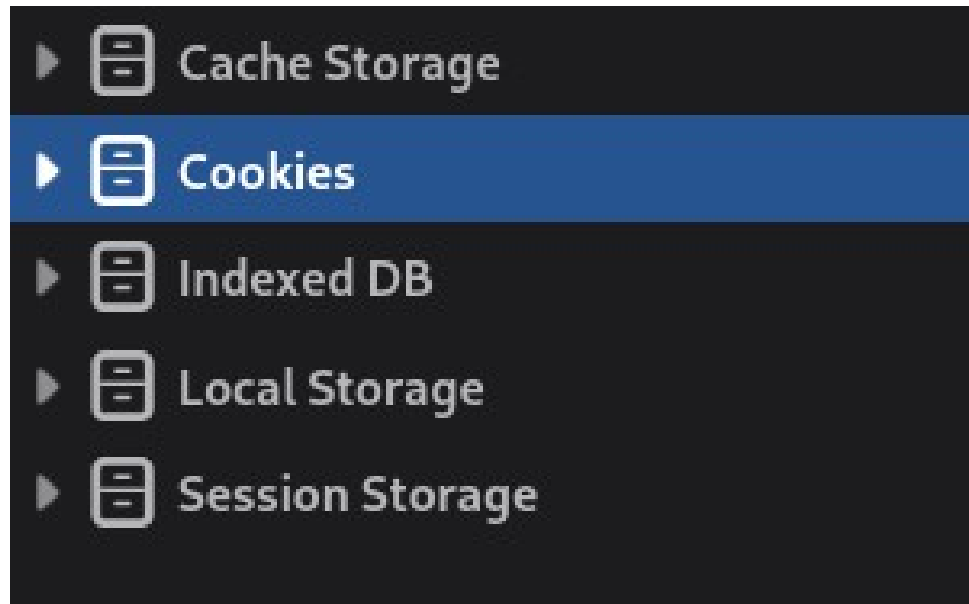


How Do We Access Cookies?



Once in the Dev Tools window, click on the Storage tab at the top of the window

How Do We Access Cookies?



The cookies are **usually** in the Cookies tab, but sometimes can be found in the Local Storage or Session Storage tab

An Example: Hack the Web

<https://hack.arrrg.de/>

Let's look at an example website to take a closer look at cookies in the Browser

Challenges: Power Cookie, Logon, and Cookie Monster Secret Recipe

The next challenges we'll cover this session is called **Power Cookie, Logon** and **Cookie Monster Secret Recipe**, which can be accessed from the master document in the Part 4 section

Hint: Remember to check for a cookie in your browser after you've logged into the web apps, and see if there's anything you can change in the cookie

Time For A Break!



We've learned a lot of web app concepts in the last couple of hours! Let's take a break! We'll resume in 20 minutes!

Part 5: HTTP Headers and HTTP Header Injection

```
GET / HTTP/2  
Host: example.com  
User-Agent: curl/8.18.0  
Accept: */*
```

What Are HTTP Headers?

Each time a web browser accesses a webpage, the browser makes an HTTP request to the server that hosts the page.

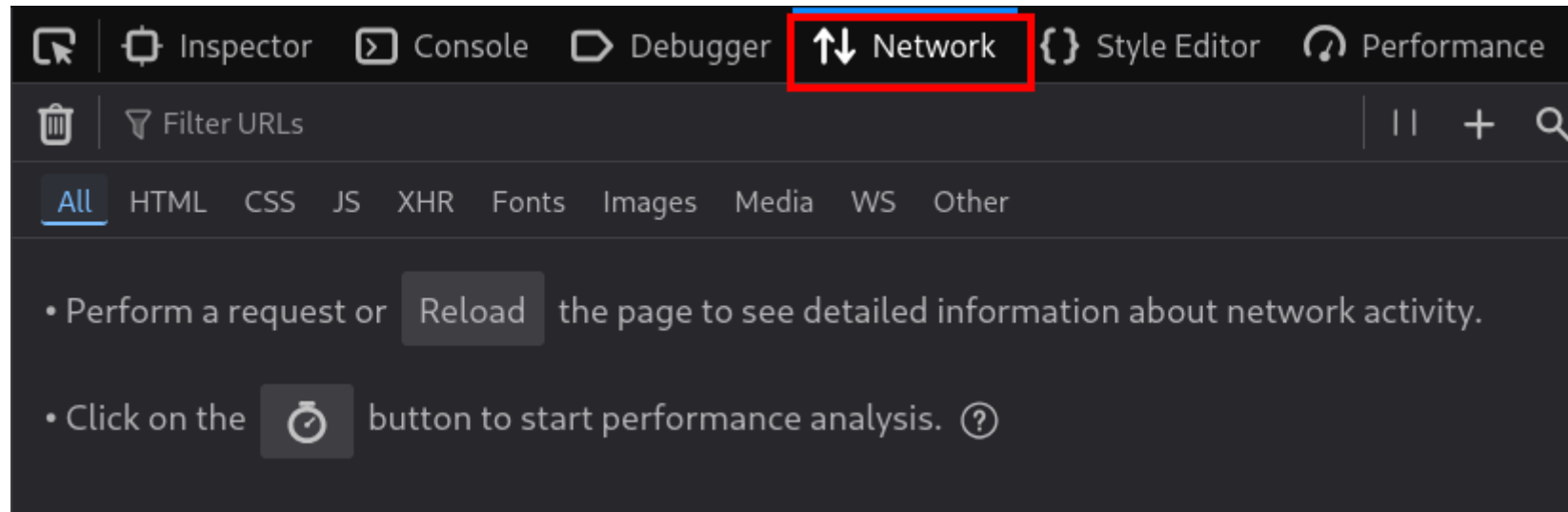


What Are HTTP Headers?

In each HTTP request, several headers and their values are passed along to the server to ensure that the browser and server can communicate properly.

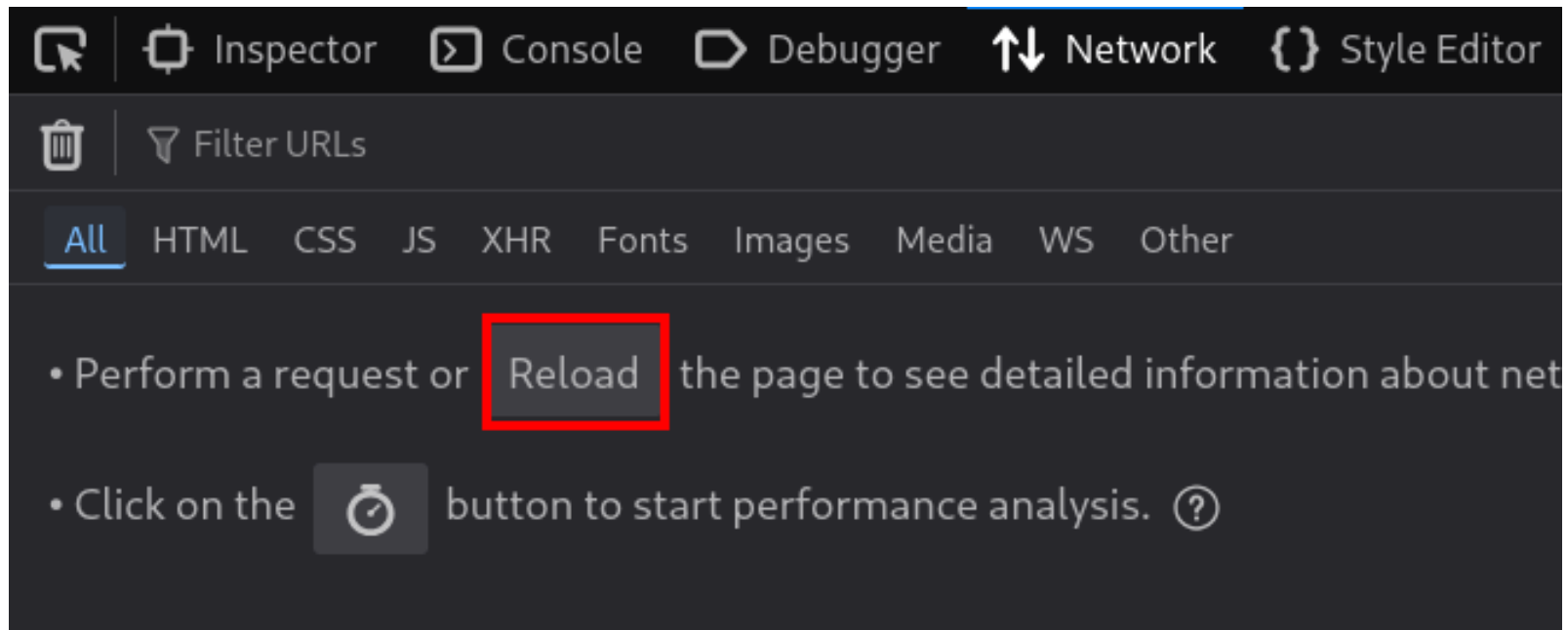


Looking At HTTP Headers In the Firefox Browser



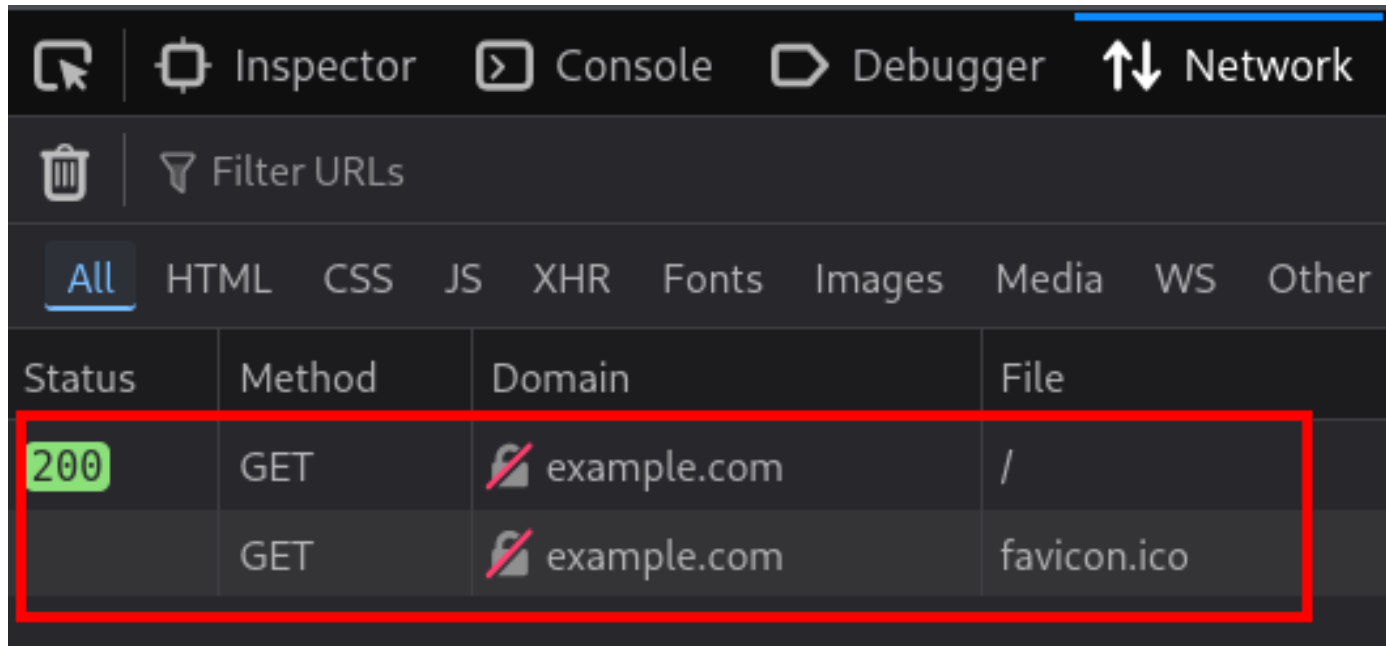
In the Firefox browser, we can access HTTP headers for all of our web requests from the Web Developer Tools, in the Network tab

Looking At HTTP Headers In the Firefox Browser



Next, click on the Reload button

Looking At HTTP Headers In the Firefox Browser

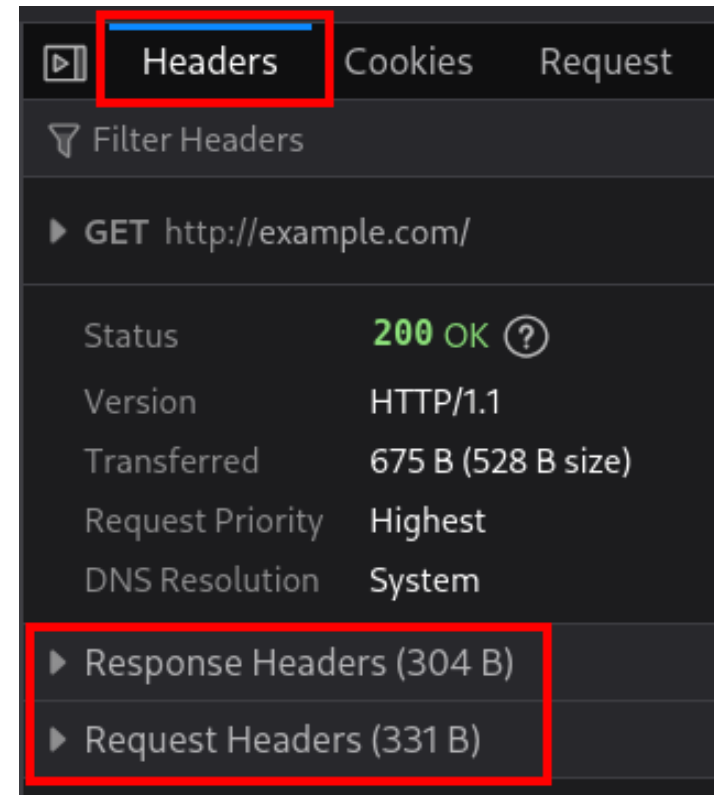


You will see a list of web requests for the current webpage

Looking At HTTP Headers In the Firefox Browser

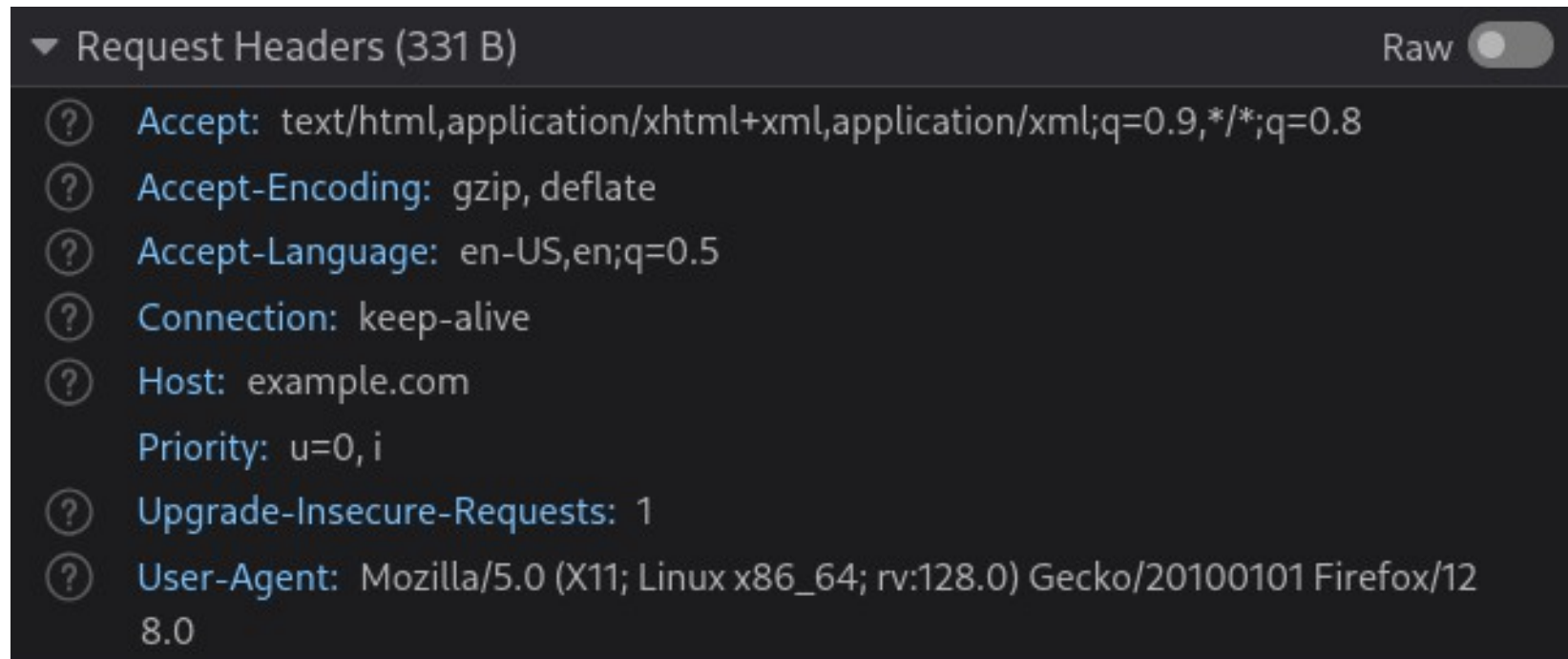
If you click on one of the requests, another window appears that contains the request details. Click the Headers tab to see the HTTP headers

Then click on the Response Headers or Request Headers tabs to see those headers



Looking At HTTP Headers

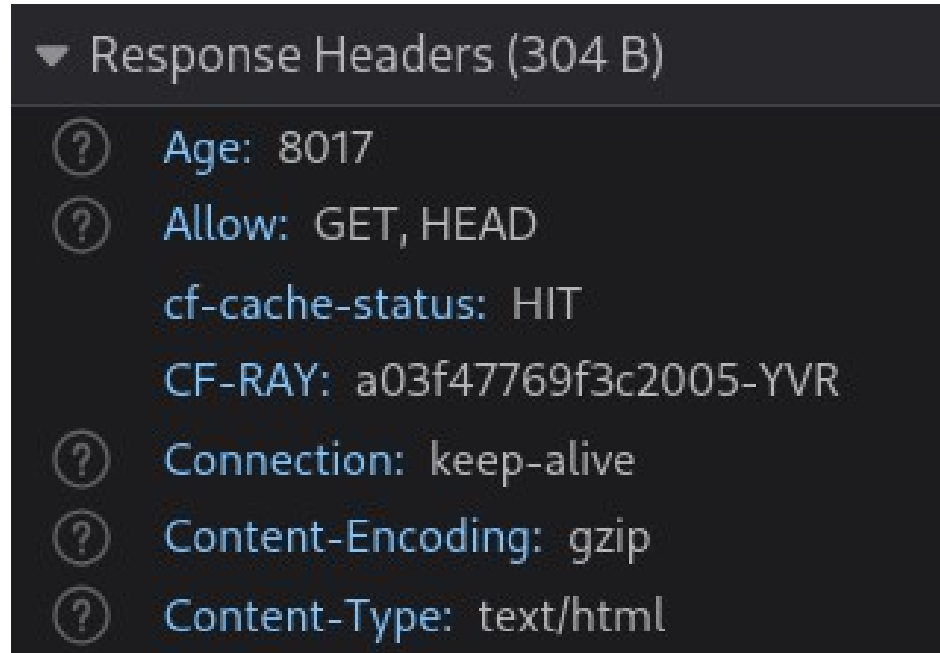
What's In the HTTP Headers?



Request Headers are what the web browser sends to the server when requesting the web page

Looking At HTTP Headers

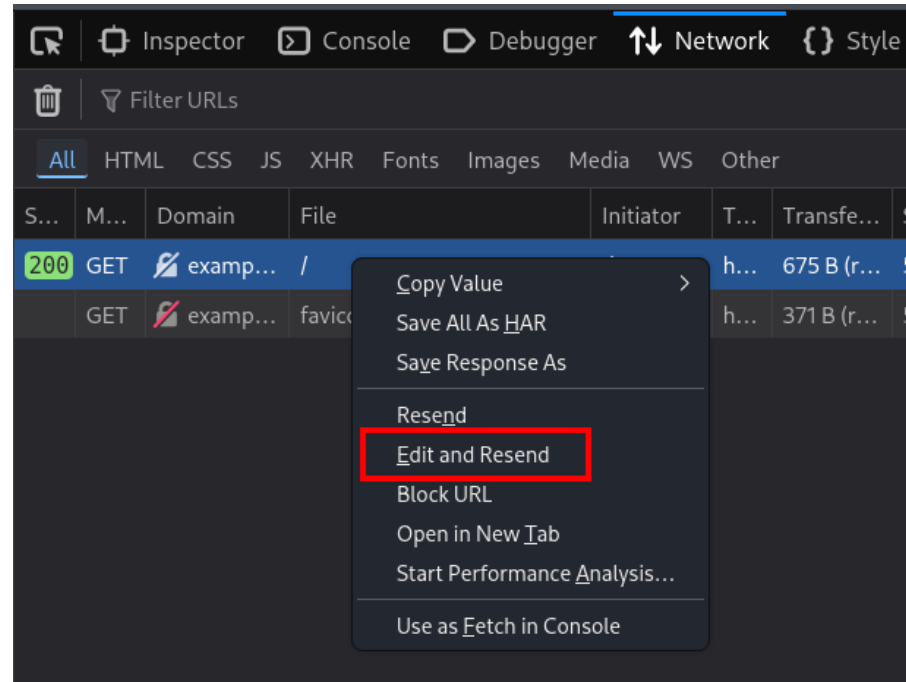
What's In the HTTP Headers?



Response Headers are what the web server sends to the web browser in response to the web request

Looking At HTTP Headers

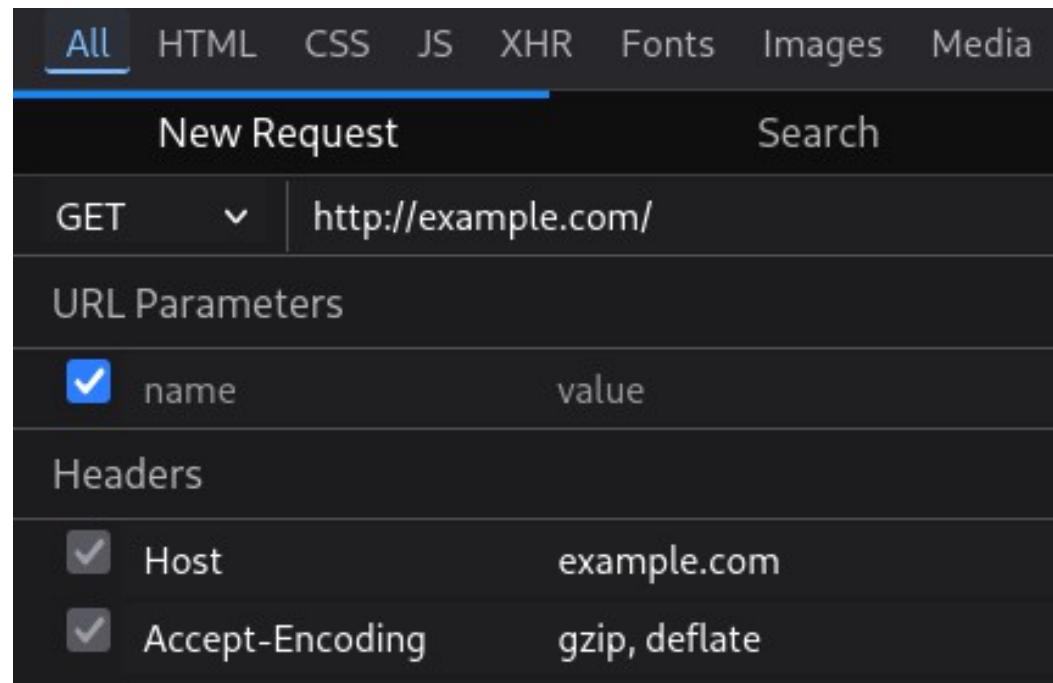
Modifying HTTP Headers



Inside the Network tab, we can right-click any request and select Edit and Resend to modify / resend the request

Looking At HTTP Headers

Modifying HTTP Headers



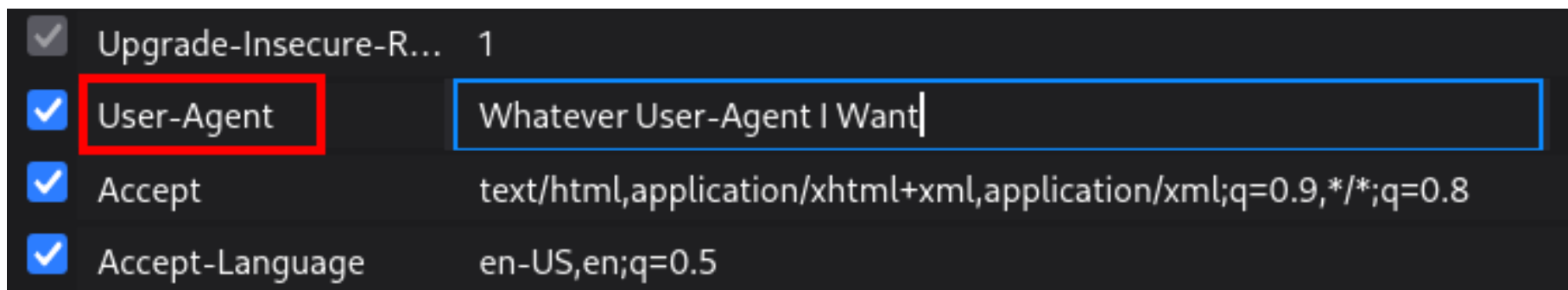
The image shows a 'New Request' dialog box from a web browser's developer tools. It has a dark theme. At the top, there are tabs for 'All', 'HTML', 'CSS', 'JS', 'XHR', 'Fonts', 'Images', and 'Media'. The 'All' tab is selected. Below the tabs, there are two buttons: 'New Request' and 'Search'. The 'New Request' button is active. Below these buttons, there is a dropdown menu showing 'GET' and a text input field containing 'http://example.com/'. Below the text input field, there is a section titled 'URL Parameters' with a table that has two columns: 'name' and 'value'. The 'name' column has a blue checkmark icon next to it. Below the 'URL Parameters' section, there is a section titled 'Headers' with a table that has two columns: 'Host' and 'example.com'. The 'Host' column has a grey checkmark icon next to it. Below the 'Headers' section, there is a table with two columns: 'Accept-Encoding' and 'gzip, deflate'. The 'Accept-Encoding' column has a grey checkmark icon next to it.

New Request		Search
GET	http://example.com/	
URL Parameters		
☒	name	value
Headers		
☒	Host	example.com
☒	Accept-Encoding	gzip, deflate

In the new window, we can modify many aspects of the web request, including HTTP headers

Looking At HTTP Headers

Modifying HTTP Headers



We can either modify an existing header--

Looking At HTTP Headers

Modifying HTTP Headers

☒	Priority	u=0, i
☒	This-Is-A-Header	Not Really
☒	name	value

Or create a whole new one

Looking At HTTP Headers

A Few Examples

- User-Agent ← The device (browser type) that is contacting the web server
- Accept ← The type of data you want to receive in the response
- Host ← The domain to the contacted

Looking At HTTP Headers

Why Do We Modify Headers?

```
GET / HTTP/2  
Host: example.com  
User-Agent: curl/8.18.0  
Accept: */*
```

Web apps send different responses depending on which headers we send, so they can be used to abuse the security of websites

Looking At HTTP Headers

HTTP Header Injection

☒	Priority	u=0, i
☒	This-Is-A-Header	Not Really
☒	name	value

When a user modifies an HTTP header on purpose to try abuse the security of a website, we call this HTTP header injection

Looking At HTTP Headers

HTTP Header Injection

☒	Upgrade-Insecure-R...	1
☒	User-Agent	Whatever User-Agent I Want
☒	Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
☒	Accept-Language	en-US,en;q=0.5

For example, a website might be configured to send a different response to the client if a specific **User-Agent** header value is used

Looking At HTTP Headers

HTTP Header Injection Practice

☒	Upgrade-Insecure-R...	1
☒	User-Agent	Whatever User-Agent I Want
☒	Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
☒	Accept-Language	en-US,en;q=0.5

Let's go to the example.com website and practice sending different HTTP headers to it

Challenges: Picobrowser and Crack the Gate 1

The next challenges we'll cover this session are called **Picobrowser**, and **Crack the Gate 1**, which can be accessed from the master document in the Part 5 section

Hint: Remember to that we can edit and resend requests with the Web Developer Tools in the Network tab, and pay attention to whether headers and values are Capitalized or not

Part 6: SQL Injection Intro



What is SQL Injection?

SQL Injection is a web app vulnerability where SQL (database) commands are passed to the web app database through insecure user input.

What is SQL Injection?

SQL databases are common components of most web apps, mostly notably, they are used to keep track of users' login credentials, usernames and passwords

What is SQL Injection?

SQL Injection vulnerabilities can lead to sensitive data exposure, admin account takeover, webserver takeover, and more.

The Most Impactful Example of SQL Injection – Login Bypass

We won't be going in-depth
Into SQL injection, because
It's a very complex topic,
But we'll introduce a
method for logging
to web apps without
knowing a set of credentials
through SQL injection



The Most Impactful Example of SQL Injection – Login Bypass

The attack is called SQL injection login bypass or authorization bypass, and it's caused by insecure web app code that trusts user input more than it should...



The Most Impactful Example of SQL Injection – Login Bypass

```
SELECT * FROM users WHERE username =  
    'user_input' AND password =  
        'user_input'
```

When we send a login request to a web app, the SQL database command looks like something similar to the one above

The Most Impactful Example of SQL Injection – Login Bypass

```
' or 1=1 --
```

The most basic auth bypass SQL command is illustrated by the SQL command shown above.

Note that there is a space present in the command after the double dashes (--).

The Most Impactful Example of SQL Injection – Login Bypass

```
SELECT * FROM users WHERE username =  
' ' or 1=1 -- ' AND password = ''
```

When we send the login bypass payload to a vulnerable web app, the SQL query is manipulated to always return true, and the password portion of the query is ignored

The Most Impactful Example of SQL Injection – Login Bypass

```
SELECT * FROM users WHERE username =  
' ' or 1=1 -- ' AND password = ''
```

When we send the login bypass payload to a vulnerable web app, the SQL query is manipulated to always return true, and the password portion of the query is ignored

SQL Login Bypass Demonstration



Let's try to use the SQL login bypass technique to break the login of a website or two. Look at the master document under Part 6 in the **Example Websites to Try To Break Into** section

Challenges: Irish Name Repo 1 and SQLite

The next challenges we'll cover this session are called **Irish Name Repo**, and **SQLite**, which can be accessed from the master document in the Part 6 section

Hint: Remember the SQL Login Bypass payload:

```
' or 1=1 -- -
```

Part 7 – Capstone Challenges



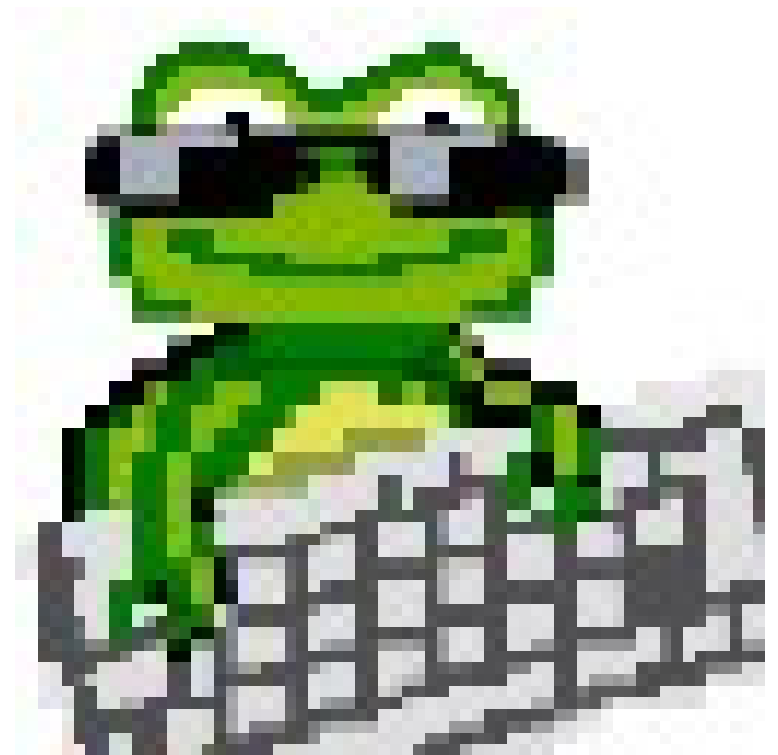
Challenges: Cookies and Scavenger Hunt

The last challenges we'll cover this session are called **Cookies**, and **Scavenger Hunt**, which can be accessed from the master document in the Part 7 section

Hint: No hints, but remember all of the techniques we've learned today, and try them out!

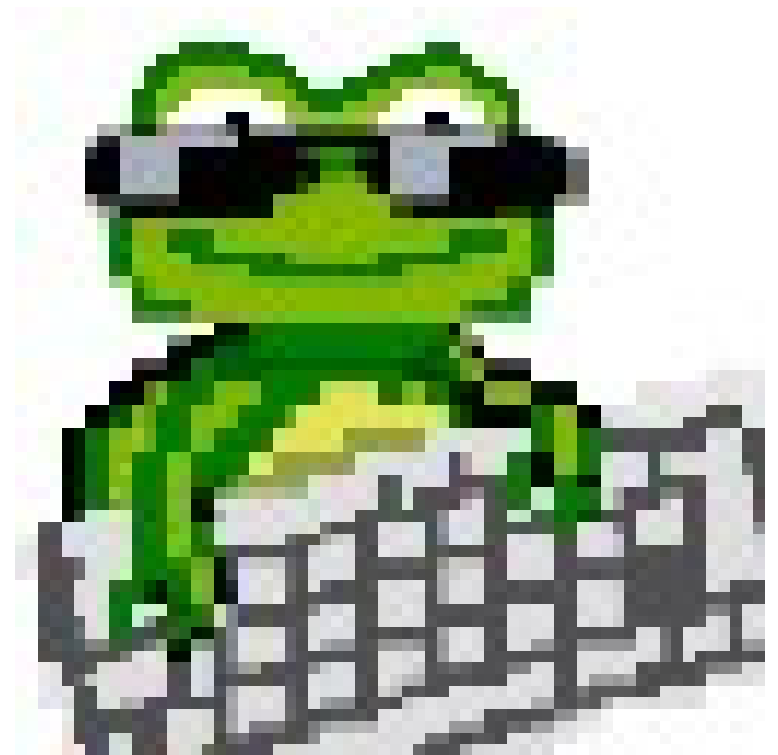
Whoami

I'm Kevin Lee, but online I go by theShyHat, and I do cybersecurity videos and live streams on platforms like YouTube and Twitch



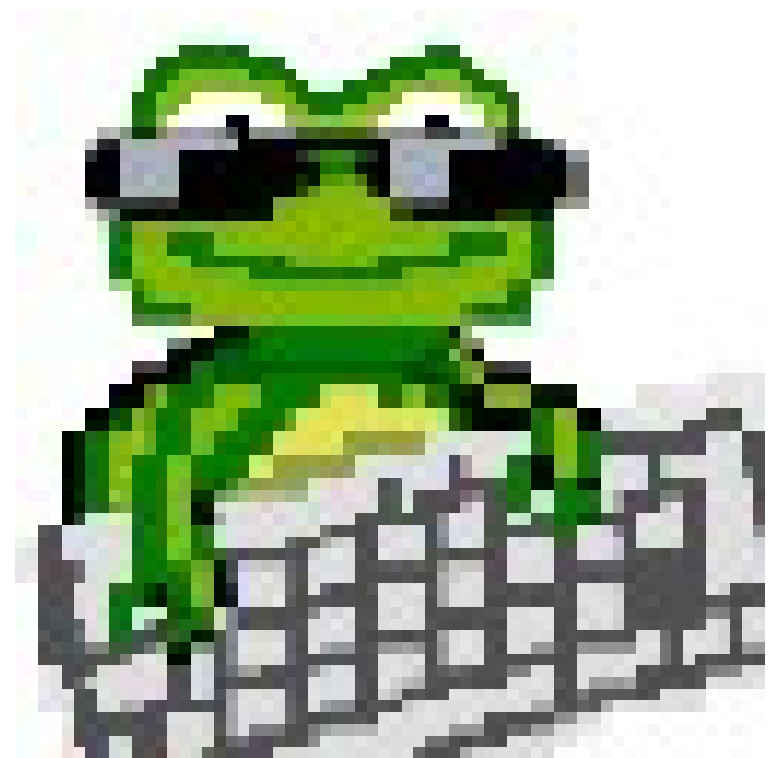
Whoami

I also teach cybersecurity
for orgs, youth programs,
and other groups looking
for training, e.g.,
Youth cybersecurity
summer camps

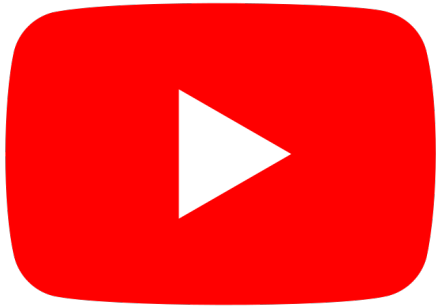


Where Do I Learn More Web Hacking?

Look at the end of the master document in the Education Links For Web Security section for a link with other websites for learning web app security



Links and Q & A



If you want to learn more cybersecurity skills with me across all sorts of topics, you can join me live on stream on Wednesdays, and Fridays

If you have any questions or comments, feel free to ask